

MÔN HỌC: CHUỖI THỜI GIAN

Biên đổi chuỗi, sai phân thường và sai phân mùa vụ

Giảng viên: Nguyễn Thị Hồng Nhung

Ngày 26 tháng 5 năm 2026

Mục lục

1	Biến đổi dữ liệu	2
1.1	Nhập dữ liệu	2
1.2	Vẽ chuỗi thời gian	3
1.3	Biến đổi log	4
1.4	Biến đổi căn bậc hai	7
1.5	Biến đổi Box–Cox	8
1.6	Sai phân bậc một	8
1.7	Sai phân bậc hai	11
1.8	Sai phân mùa vụ	14
2	Thực hành	19
2.1	Ví dụ mô phỏng dữ liệu giáo dục	19
2.2	Bài tập thực hành 1.	20
2.3	Bài tập thực hành 2.	20

Mục tiêu thực hành

1. Nhận biết khi nào cần biến đổi dữ liệu chuỗi thời gian.
2. Thực hiện các biến đổi thường dùng: log, căn bậc hai và Box–Cox.
3. Thực hiện sai phân bậc một, sai phân bậc hai và sai phân mùa vụ.
4. Giải thích ý nghĩa của sai phân log trong phân tích tốc độ tăng trưởng.
5. So sánh chuỗi trước và sau biến đổi bằng đồ thị.
6. Sử dụng ACF để đánh giá sơ bộ tính dừng sau khi biến đổi.
7. Thực hành trên dữ liệu có sẵn trong R và dữ liệu mô phỏng trong giáo dục.

Trọng tâm Ở các phần trước, chúng ta đã học cách nhận diện xu hướng, mùa vụ, ACF, PACF và tính dừng. Bài thực hành này tập trung vào việc biến đổi dữ liệu để làm cho chuỗi phù hợp hơn với mô hình hóa, đặc biệt là chuẩn bị cho ARIMA và SARIMA.

Chuẩn bị môi trường R

Trong bài thực hành này, ta sử dụng các gói chính sau:

```
library(ggplot2)
library(forecast)
```

Nếu máy chưa có gói, có thể cài đặt trước bằng:

```
install.packages("ggplot2")
install.packages("forecast")
```

Ta viết một hàm nhỏ để chuyển đối tượng `ts` sang `data.frame`, thuận tiện cho việc vẽ bằng `ggplot2`.

```
ts_to_df <- function(x, name = "value") {
  data.frame(
    time = as.numeric(time(x)),
    value = as.numeric(x)
  )
}
```

1 Biến đổi dữ liệu

1.1 Nhập dữ liệu

Bộ dữ liệu `AirPassengers` có sẵn trong R. Đây là chuỗi số lượng hành khách hàng không quốc tế theo tháng từ năm 1949 đến năm 1960.

```
data("AirPassengers")

ap <- AirPassengers
ap
```

```
##      Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
## 1949 112 118 132 129 121 135 148 148 136 119 104 118
## 1950 115 126 141 135 125 149 170 170 158 133 114 140
## 1951 145 150 178 163 172 178 199 199 184 162 146 166
## 1952 171 180 193 181 183 218 230 242 209 191 172 194
## 1953 196 196 236 235 229 243 264 272 237 211 180 201
## 1954 204 188 235 227 234 264 302 293 259 229 203 229
## 1955 242 233 267 269 270 315 364 347 312 274 237 278
## 1956 284 277 317 313 318 374 413 405 355 306 271 306
## 1957 315 301 356 348 355 422 465 467 404 347 305 336
## 1958 340 318 362 348 363 435 491 505 404 359 310 337
## 1959 360 342 406 396 420 472 548 559 463 407 362 405
## 1960 417 391 419 461 472 535 622 606 508 461 390 432
```

Thông tin cơ bản của chuỗi:

```
start(ap)
```

```
## [1] 1949    1
```

```
end(ap)
```

```
## [1] 1960   12
```

```
frequency(ap)
```

```
## [1] 12
```

```
length(ap)
```

```
## [1] 144
```

Trong đó:

- `start(ap)` cho biết thời điểm bắt đầu của chuỗi.
- `end(ap)` cho biết thời điểm kết thúc của chuỗi.
- `frequency(ap) = 12` cho biết dữ liệu được ghi nhận theo tháng.
- `length(ap)` là số lượng quan sát.

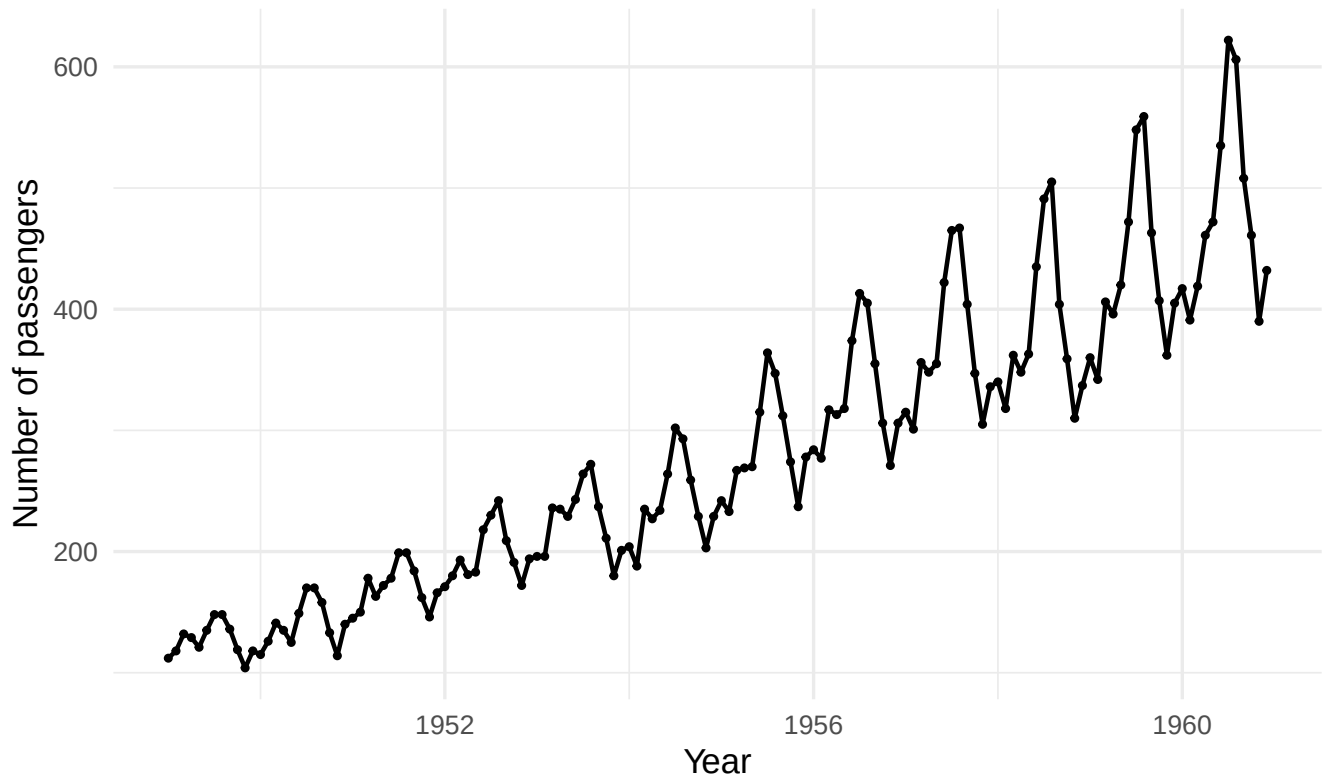
1.2 Vẽ chuỗi thời gian

```
ap_df <- ts_to_df(ap)

ggplot(ap_df, aes(x = time, y = value)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 0.8) +
  labs(
    title = "AirPassengers: original series",
    x = "Year",
```

```
y = "Number of passengers"
) +
theme_minimal(base_size = 13)
```

AirPassengers: original series



Quan sát đồ thị trên và trả lời:

1. Chuỗi có xu hướng tăng hay không?
2. Chuỗi có thành phần mùa vụ hay không?
3. Biên độ dao động mùa vụ có tăng theo thời gian hay không?
4. Phương sai của chuỗi có vẻ ổn định hay không?
5. Vì sao chuỗi này thường được biến đổi log trước khi mô hình hóa?

1.3 Biến đổi log

Biến đổi log thường được dùng khi chuỗi có phương sai tăng theo thời gian hoặc có cấu trúc nhân. Nếu chuỗi gốc được mô tả gần đúng bởi

$$Y_t = T_t \times S_t \times W_t,$$

thì lấy log cho ta

$$\log(Y_t) = \log(T_t) + \log(S_t) + \log(W_t).$$

Như vậy, biến đổi log giúp chuyển mô hình nhân thành mô hình cộng.

Thực hiện biến đổi log trong R

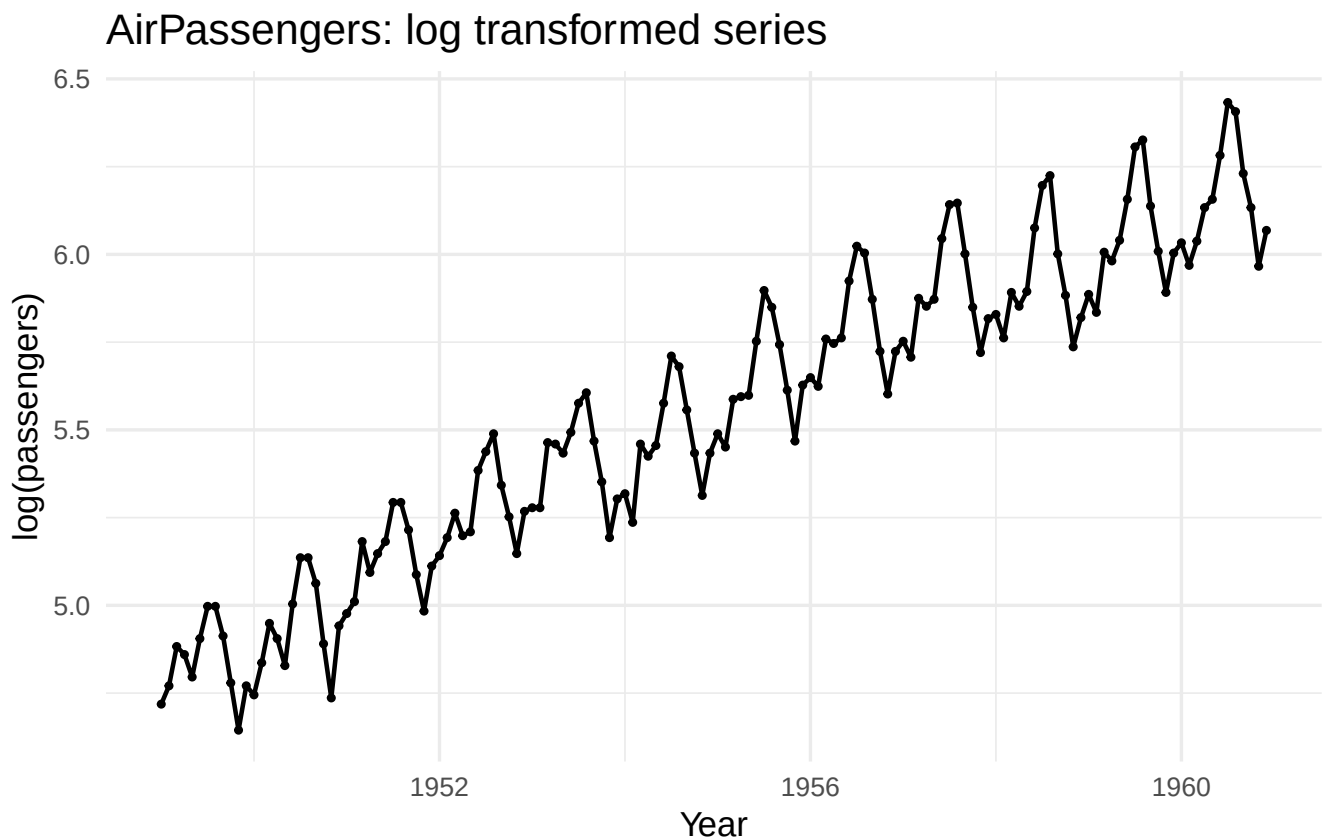
```
log_ap <- log(ap)
head(log_ap)
```

```
##           Jan      Feb      Mar      Apr      May      Jun
## 1949 4.718499 4.770685 4.882802 4.859812 4.795791 4.905275
```

Vẽ chuỗi sau khi lấy log:

```
log_ap_df <- ts_to_df(log_ap)

ggplot(log_ap_df, aes(x = time, y = value)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 0.8) +
  labs(
    title = "AirPassengers: log transformed series",
    x = "Year",
    y = "log(passengers)"
  ) +
  theme_minimal(base_size = 13)
```



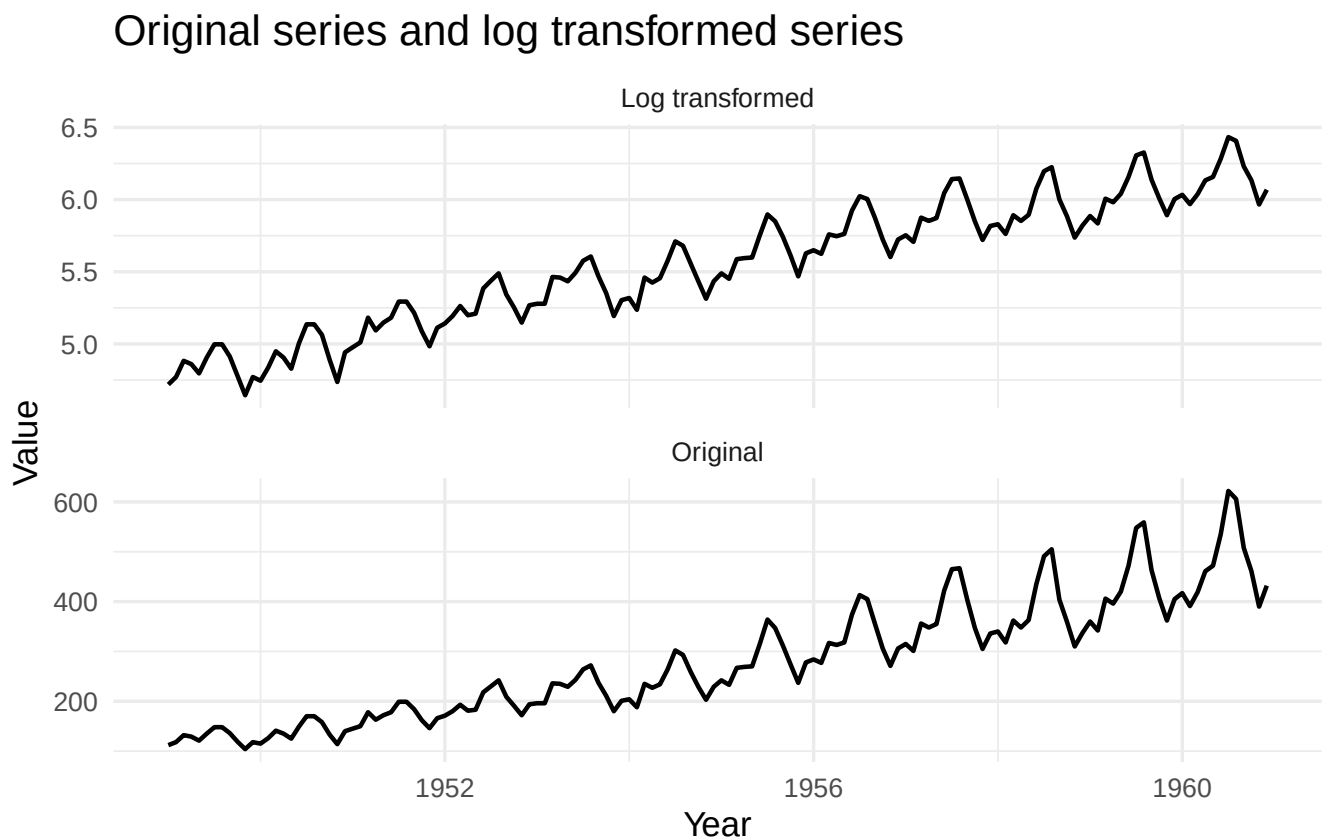
So sánh chuỗi gốc và chuỗi log

```

ap_compare <- rbind(
  data.frame(time = as.numeric(time(ap)), value = as.numeric(ap), type = "Original"),
  data.frame(time = as.numeric(time(log_ap)), value = as.numeric(log_ap), type = "Log t
)

ggplot(ap_compare, aes(x = time, y = value)) +
  geom_line(linewidth = 0.8) +
  facet_wrap(~ type, scales = "free_y", ncol = 1) +
  labs(
    title = "Original series and log transformed series",
    x = "Year",
    y = "Value"
  ) +
  theme_minimal(base_size = 13)

```



Sau biến đổi log, biên độ dao động mùa vụ trở nên ổn định hơn. Tuy nhiên, chuỗi vẫn còn xu hướng tăng và mùa vụ rõ rệt. Điều này cho thấy biến đổi log chỉ giúp ổn định phương sai, chưa đủ để làm chuỗi dừng.

Câu hỏi thảo luận. Sau khi lấy log, cần tiếp tục thực hiện bước nào để xử lý xu hướng hoặc mùa vụ?

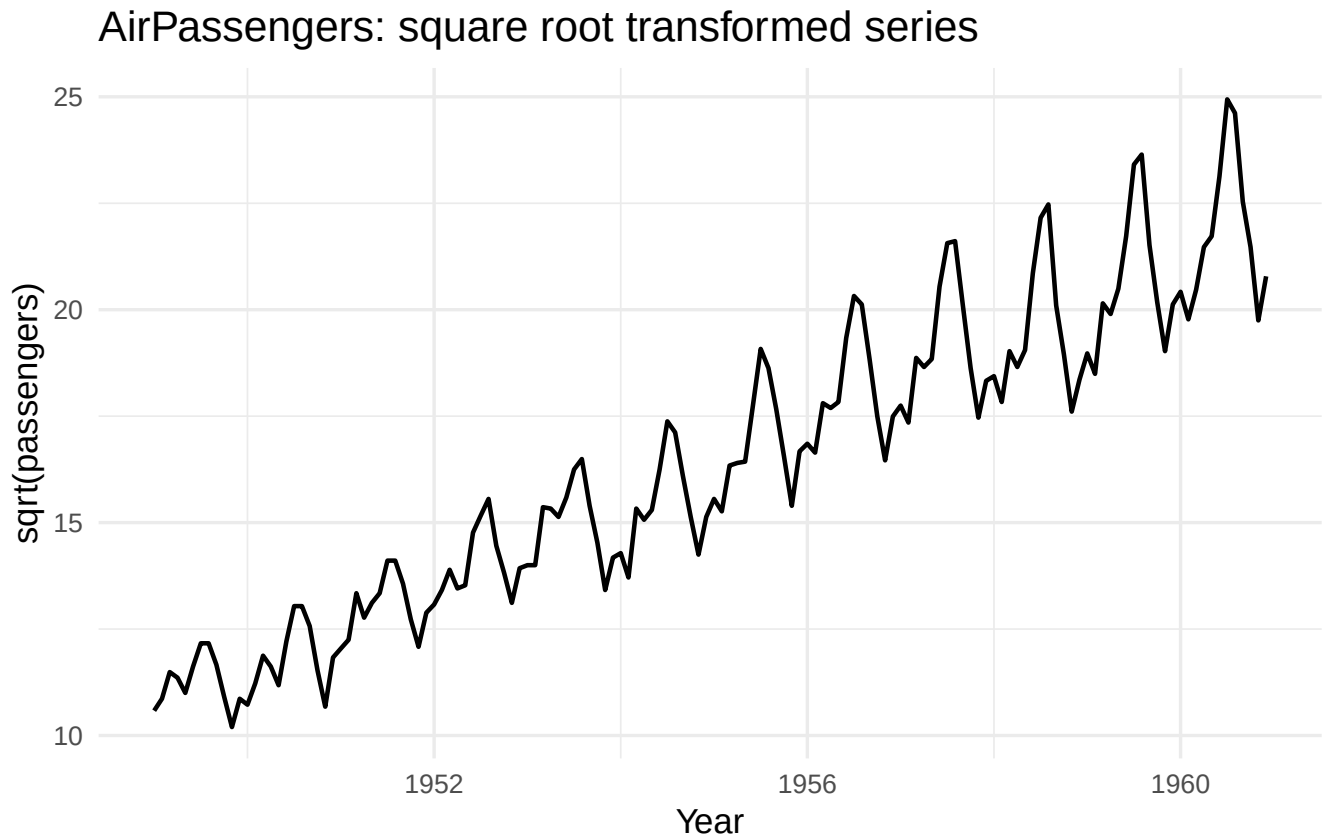
1.4 Biến đổi căn bậc hai

Biến đổi căn bậc hai cũng được dùng để giảm mức độ biến động, nhất là với dữ liệu đếm. Tuy nhiên, so với log, căn bậc hai thường có tác động nhẹ hơn.

```
sqrt_ap <- sqrt(ap)
```

```
sqrt_ap_df <- ts_to_df(sqrt_ap)
```

```
ggplot(sqrt_ap_df, aes(x = time, y = value)) +  
  geom_line(linewidth = 0.8) +  
  labs(  
    title = "AirPassengers: square root transformed series",  
    x = "Year",  
    y = "sqrt(passengers)"  
  ) +  
  theme_minimal(base_size = 13)
```



So sánh ba chuỗi sau:

1. Chuỗi gốc ap.
2. Chuỗi $\log(ap)$.
3. Chuỗi $\sqrt{ap}$.

Câu hỏi:

- Biến đổi nào làm biên độ dao động ổn định hơn?
- Biến đổi nào có tác động mạnh hơn?
- Trong trường hợp này, em chọn log hay căn bậc hai? Vì sao?

1.5 Biến đổi Box–Cox

Công thức Box–Cox

Biến đổi Box–Cox được định nghĩa bởi

$$Y_t^{(\lambda)} = \begin{cases} \frac{Y_t^\lambda - 1}{\lambda}, & \lambda \neq 0, \\ \log(Y_t), & \lambda = 0. \end{cases}$$

Trong thực hành, tham số λ có thể được ước lượng từ dữ liệu.

**** Ước lượng tham số Box–Cox ****

```
lambda_ap <- BoxCox.lambda(ap)
lambda_ap
```

```
## [1] -0.2947156
```

Thực hiện biến đổi Box–Cox:

```
bc_ap <- BoxCox(ap, lambda = lambda_ap)
```

```
bc_ap_df <- ts_to_df(bc_ap)
```

```
ggplot(bc_ap_df, aes(x = time, y = value)) +
  geom_line(linewidth = 0.8) +
  labs(
    title = "AirPassengers: Box-Cox transformed series",
    x = "Year",
    y = "Box-Cox transformed value"
  ) +
  theme_minimal(base_size = 13)
```

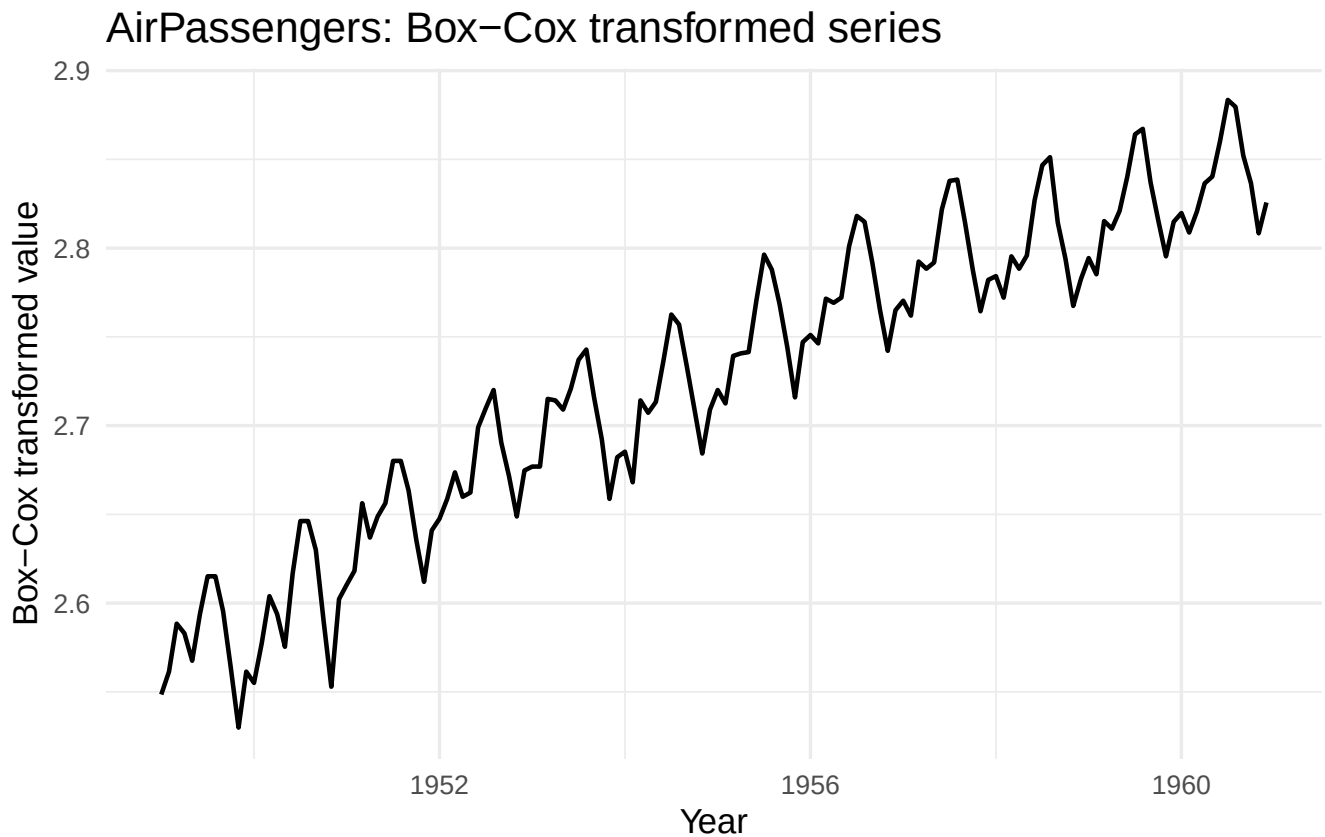
Nếu λ gần 0, biến đổi Box–Cox gần với biến đổi log. Nếu $\lambda = 1$, gần như không biến đổi chuỗi. Nếu $\lambda = 0.5$, biến đổi gần với căn bậc hai.

Câu hỏi. Với dữ liệu `AirPassengers`, giá trị λ ước lượng được gần với biến đổi nào?

1.6 Sai phân bậc một

Khái niệm

Sai phân bậc một được định nghĩa bởi



$$\nabla Y_t = Y_t - Y_{t-1}.$$

Sai phân bậc một thường dùng để loại bỏ xu hướng tuyến tính hoặc xu hướng ngẫu nhiên.

Sai phân chuỗi log

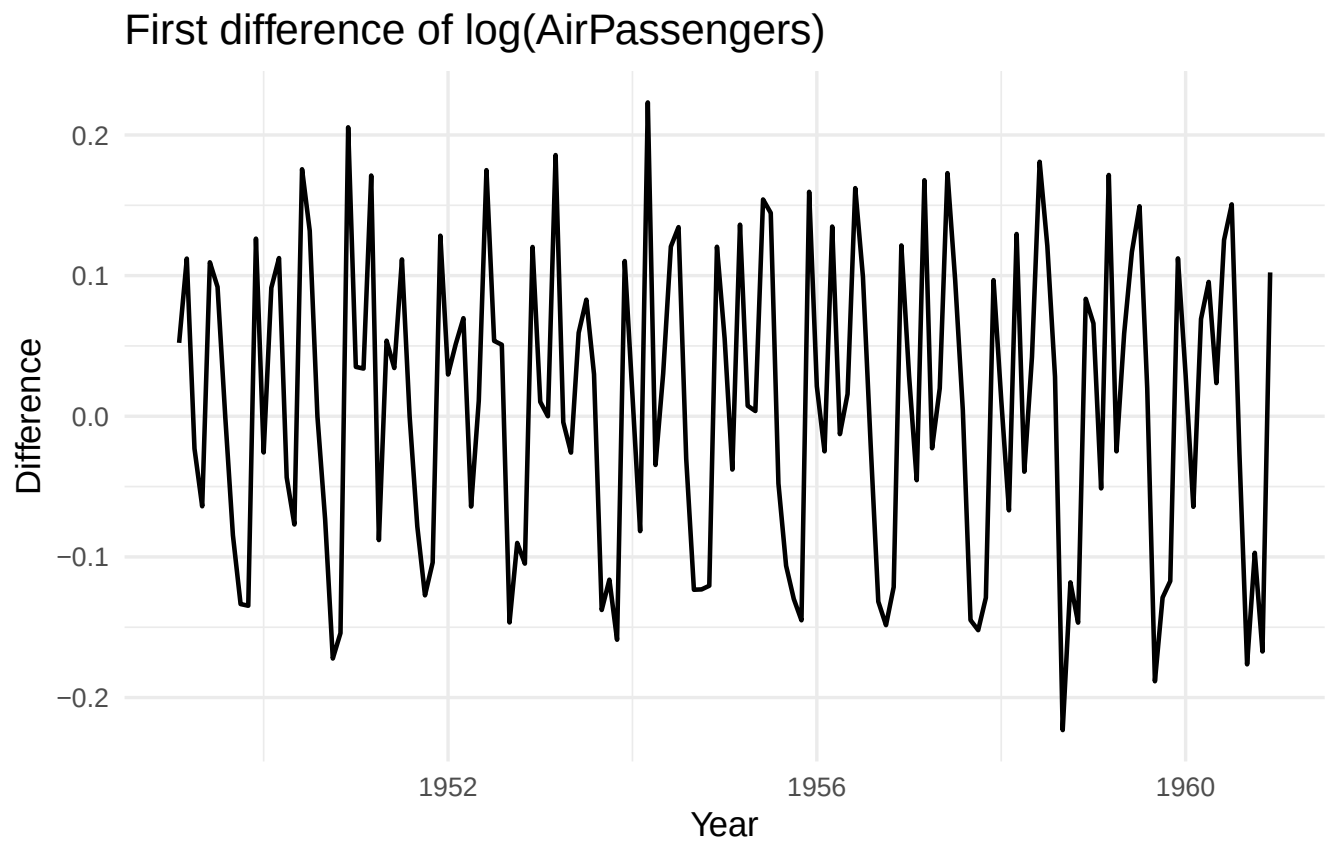
Ta áp dụng sai phân bậc một cho chuỗi đã lấy log:

```
diff_log_ap <- diff(log_ap, differences = 1)
```

```
diff_log_ap_df <- ts_to_df(diff_log_ap)
```

```
ggplot(diff_log_ap_df, aes(x = time, y = value)) +
  geom_line(linewidth = 0.8) +
  labs(
    title = "First difference of log(AirPassengers)",
    x = "Year",
    y = "Difference"
  ) +
  theme_minimal(base_size = 13)
```

Với chuỗi dương, sai phân log



Hình 1: Sai phân bậc một của chuỗi $\log(\text{AirPassengers})$.

$$\log(Y_t) - \log(Y_{t-1})$$

xấp xỉ tốc độ tăng trưởng tương đối từ thời điểm $t - 1$ sang thời điểm t .

Cụ thể,

$$100[\log(Y_t) - \log(Y_{t-1})]$$

có thể được hiểu xấp xỉ là phần trăm tăng trưởng.

```
growth_ap <- 100 * diff(log_ap)
summary(as.numeric(growth_ap))

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -22.314  -8.002   1.482   0.944  10.588  22.314

growth_ap_df <- ts_to_df(growth_ap)

ggplot(growth_ap_df, aes(x = time, y = value)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(
    title = "Approximate monthly growth rate",
    x = "Year",
    y = "Growth rate (%)"
  ) +
  theme_minimal(base_size = 13)
```

1. Chuỗi sai phân log còn xu hướng tăng rõ rệt không?
2. Chuỗi sai phân log còn mùa vụ không?
3. Tốc độ tăng trưởng tháng nào có thể âm?
4. Vì sao sai phân log thường được dùng trong kinh tế và tài chính?

1.7 Sai phân bậc hai

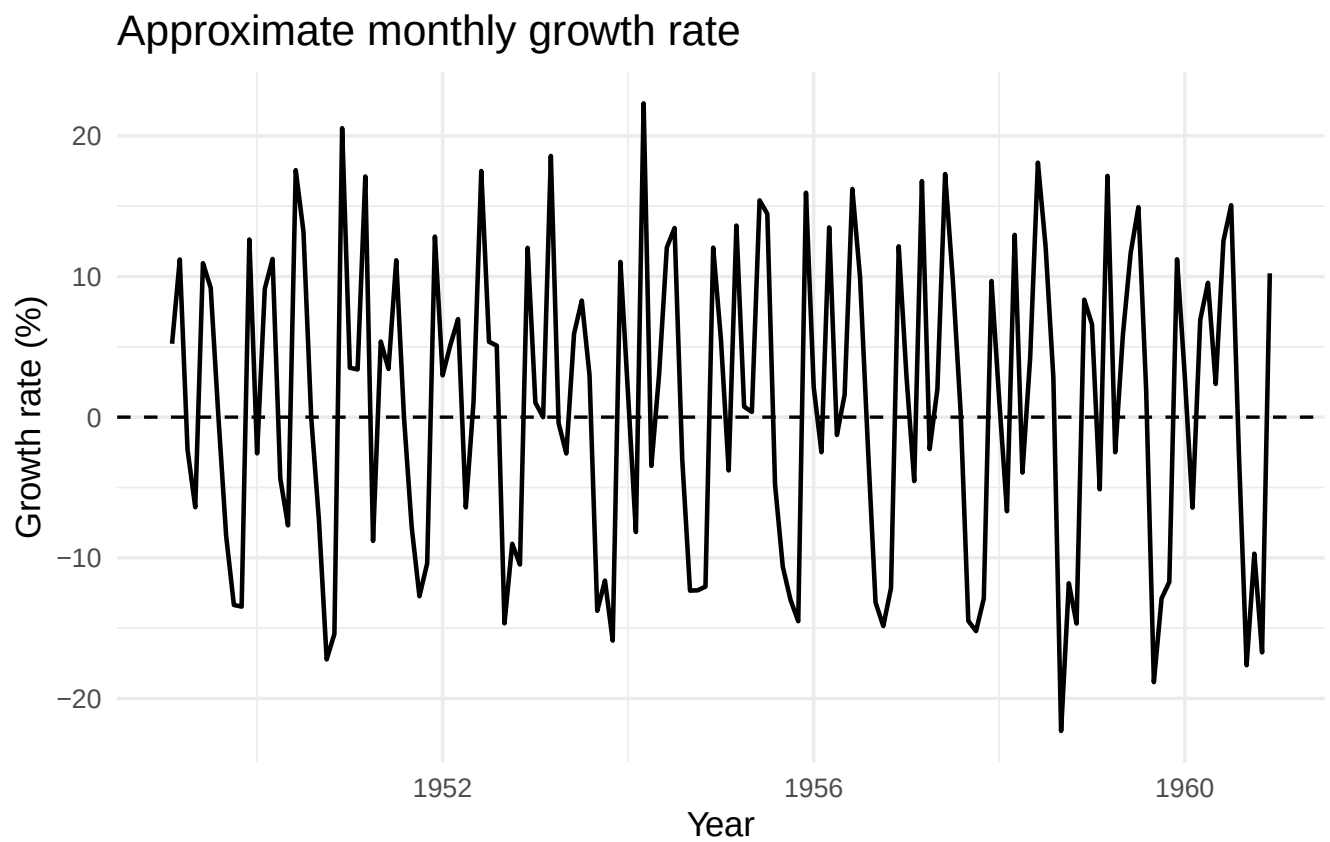
Sai phân bậc hai

Sai phân bậc hai được định nghĩa bởi

$$\nabla^2 Y_t = \nabla(\nabla Y_t).$$

Trong R, có thể dùng:

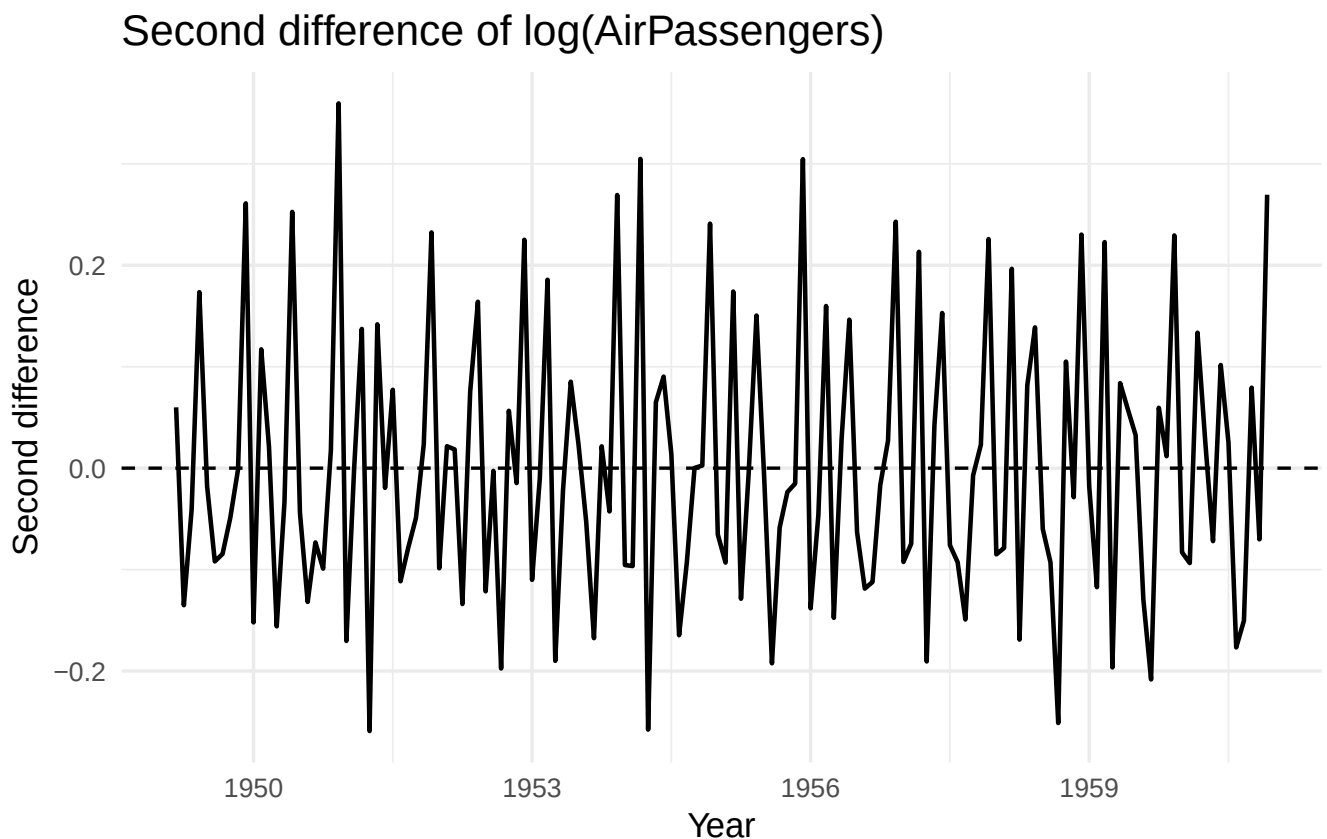
```
diff2_log_ap <- diff(log_ap, differences = 2)
```



Hình 2: Tốc độ tăng trưởng xấp xỉ của AirPassengers theo tháng.

```
diff2_log_ap_df <- ts_to_df(diff2_log_ap)

ggplot(diff2_log_ap_df, aes(x = time, y = value)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(
    title = "Second difference of log(AirPassengers)",
    x = "Year",
    y = "Second difference"
  ) +
  theme_minimal(base_size = 13)
```



Hình 3: Sai phân bậc hai của chuỗi $\log(\text{AirPassengers})$.

Cảnh báo về quá sai phân

Không phải cứ sai phân nhiều lần thì mô hình sẽ tốt hơn. Sai phân quá mức có thể làm mất thông tin quan trọng và tạo ra cấu trúc phụ thuộc không cần thiết.

Lưu ý. Trong ARIMA, tham số d thường là 0, 1 hoặc đôi khi là 2. Nếu phải sai phân quá nhiều lần, cần xem lại dữ liệu, cách biến đổi hoặc giả định mô hình.

1.8 Sai phân mùa vụ

Khái niệm

Với dữ liệu theo tháng, sai phân mùa vụ bậc 12 được định nghĩa bởi

$$\nabla_{12}Y_t = Y_t - Y_{t-12}.$$

Sai phân mùa vụ giúp loại bỏ cấu trúc lặp lại theo năm trong dữ liệu tháng.

Sai phân mùa vụ cho chuỗi `log(AirPassengers)`

```
seasonal_diff_log_ap <- diff(log_ap, lag = 12, differences = 1)

seasonal_diff_log_ap_df <- ts_to_df(seasonal_diff_log_ap)

ggplot(seasonal_diff_log_ap_df, aes(x = time, y = value)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(
    title = "Seasonal difference of log(AirPassengers)",
    x = "Year",
    y = "Seasonal difference"
  ) +
  theme_minimal(base_size = 13)
```

Kết hợp sai phân thường và sai phân mùa vụ

Trong nhiều chuỗi có cả xu hướng và mùa vụ, ta có thể kết hợp:

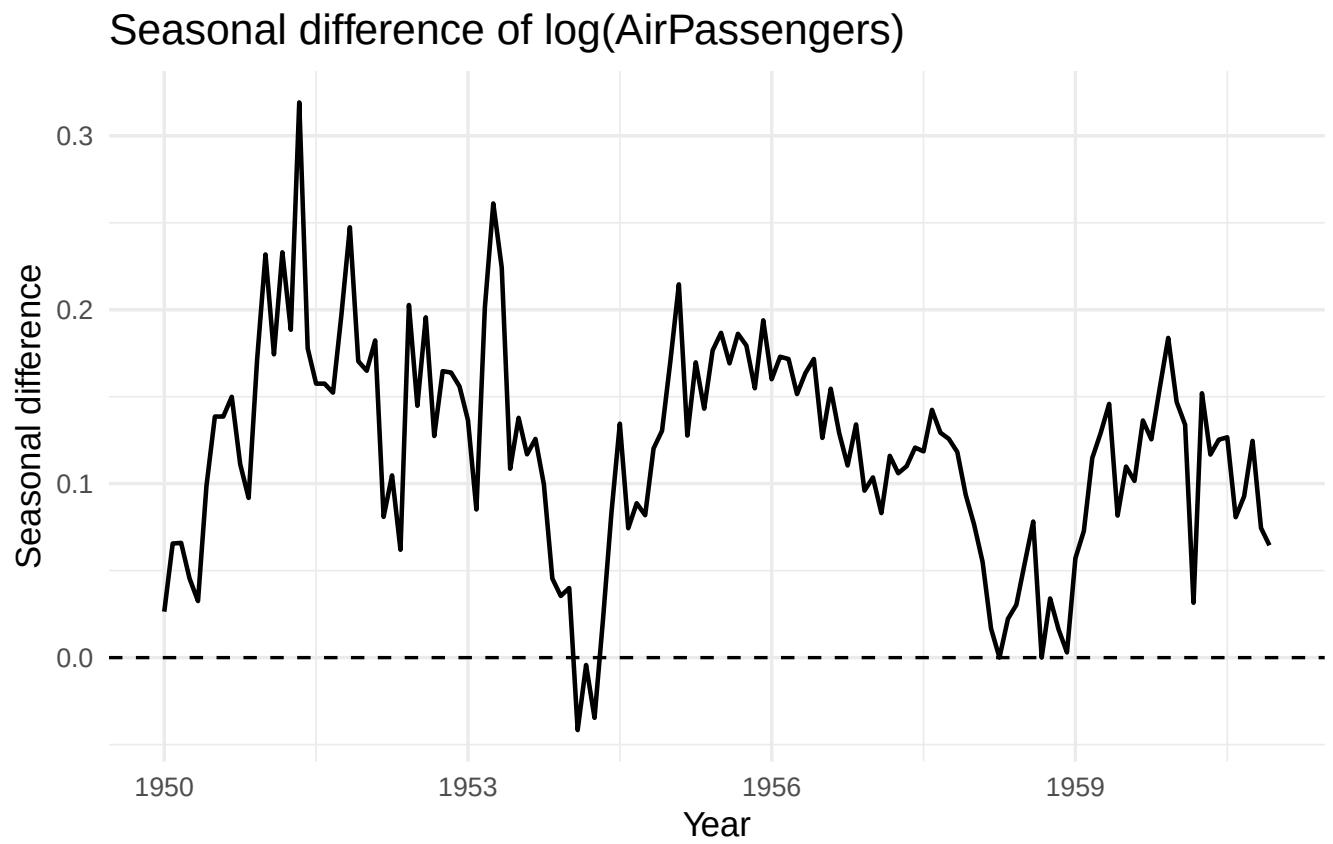
$$\nabla \nabla_{12} \log(Y_t).$$

```
combined_diff_log_ap <- diff(diff(log_ap, lag = 12), differences = 1)

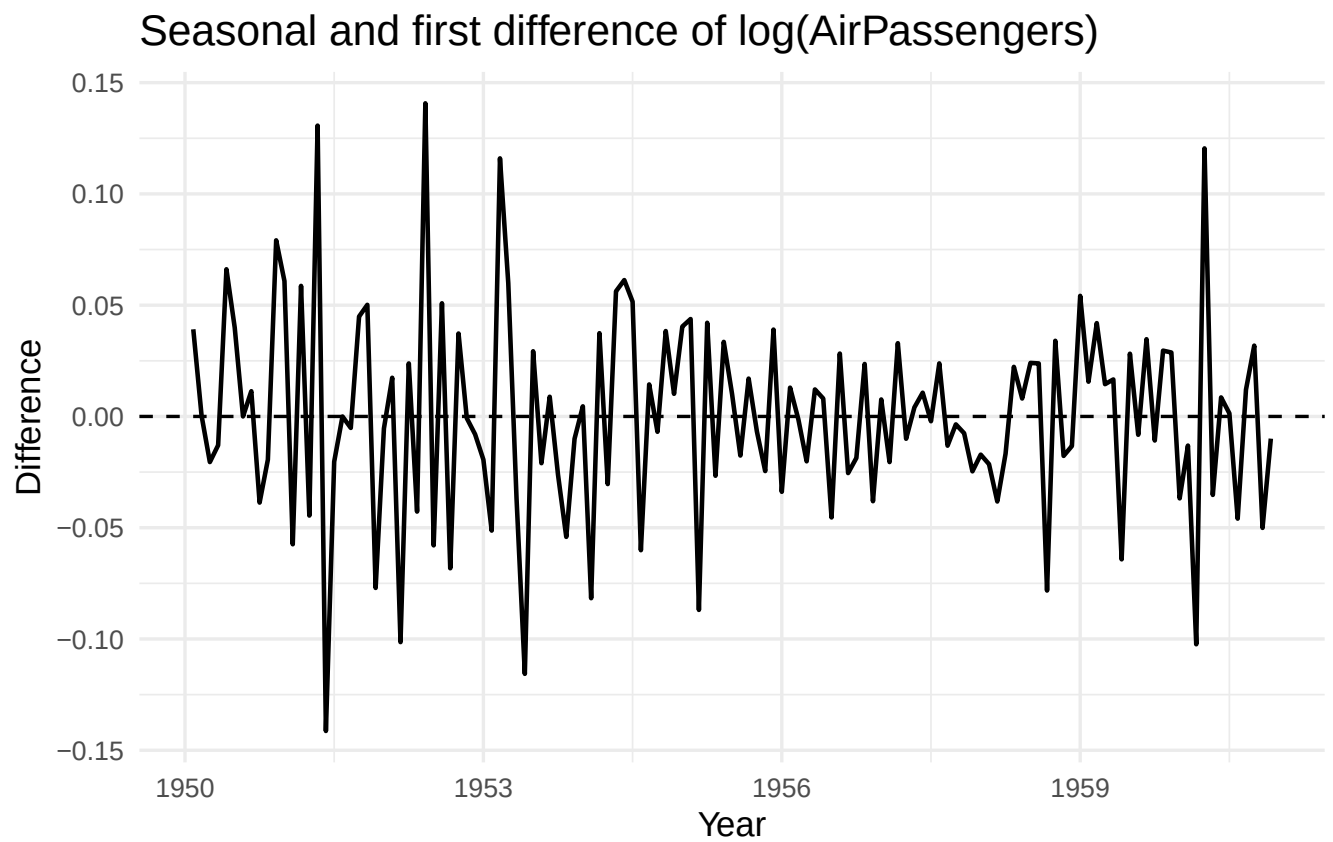
combined_diff_log_ap_df <- ts_to_df(combined_diff_log_ap)

ggplot(combined_diff_log_ap_df, aes(x = time, y = value)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(
    title = "Seasonal and first difference of log(AirPassengers)",
    x = "Year",
    y = "Difference"
  ) +
  theme_minimal(base_size = 13)
```

1. Sai phân mùa vụ làm giảm thành phần nào trong chuỗi?



Hình 4: Sai phân mùa vụ của chuỗi $\log(\text{AirPassengers})$.



Hình 5: Kết hợp sai phân mùa vụ và sai phân thường.

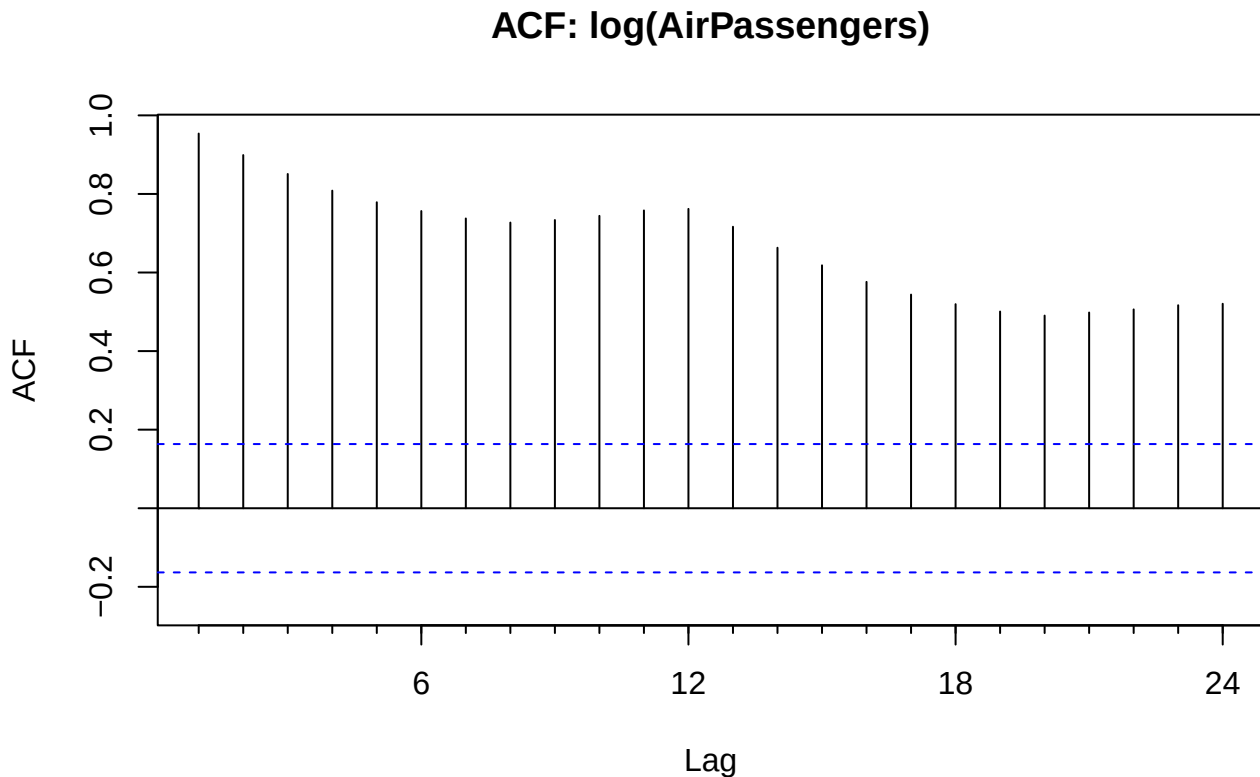
2. Sai phân thường làm giảm thành phần nào trong chuỗi?
3. Vì sao với `AirPassengers` nên lấy log trước khi sai phân?
4. Sau khi kết hợp sai phân mùa vụ và sai phân thường, chuỗi có vẻ dao động quanh trung bình ổn định hơn không?

**** So sánh ACF trước và sau sai phân****

ACF giúp ta quan sát mức độ phụ thuộc theo độ trễ. Chuỗi có xu hướng hoặc mùa vụ thường có ACF giảm chậm hoặc có đỉnh tại các độ trễ mùa vụ.

ACF của chuỗi log gốc

```
Acf(log_ap, main = "ACF: log(AirPassengers)")
```



Hình 6: ACF của chuỗi `log(AirPassengers)`.

ACF của chuỗi sau sai phân bậc một

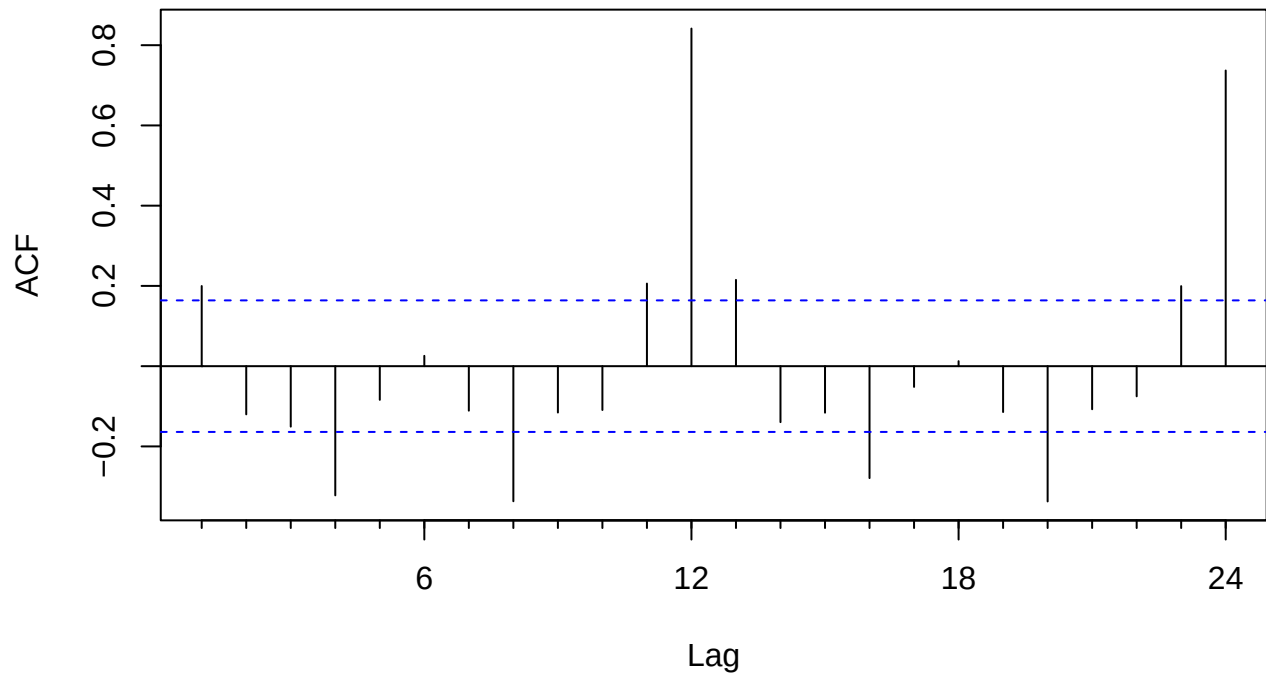
```
Acf(diff_log_ap, main = "ACF: first difference")
```

ACF của chuỗi sau sai phân mùa vụ

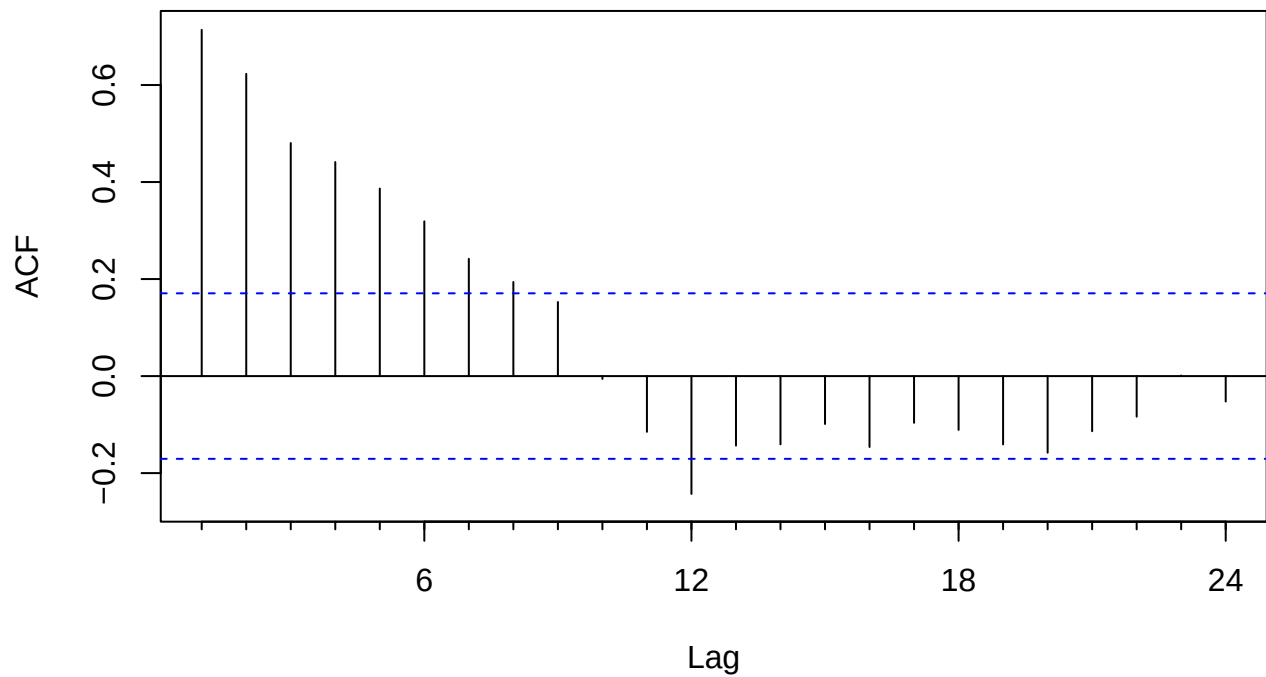
```
Acf(seasonal_diff_log_ap, main = "ACF: seasonal difference")
```

ACF của chuỗi sau khi kết hợp hai loại sai phân

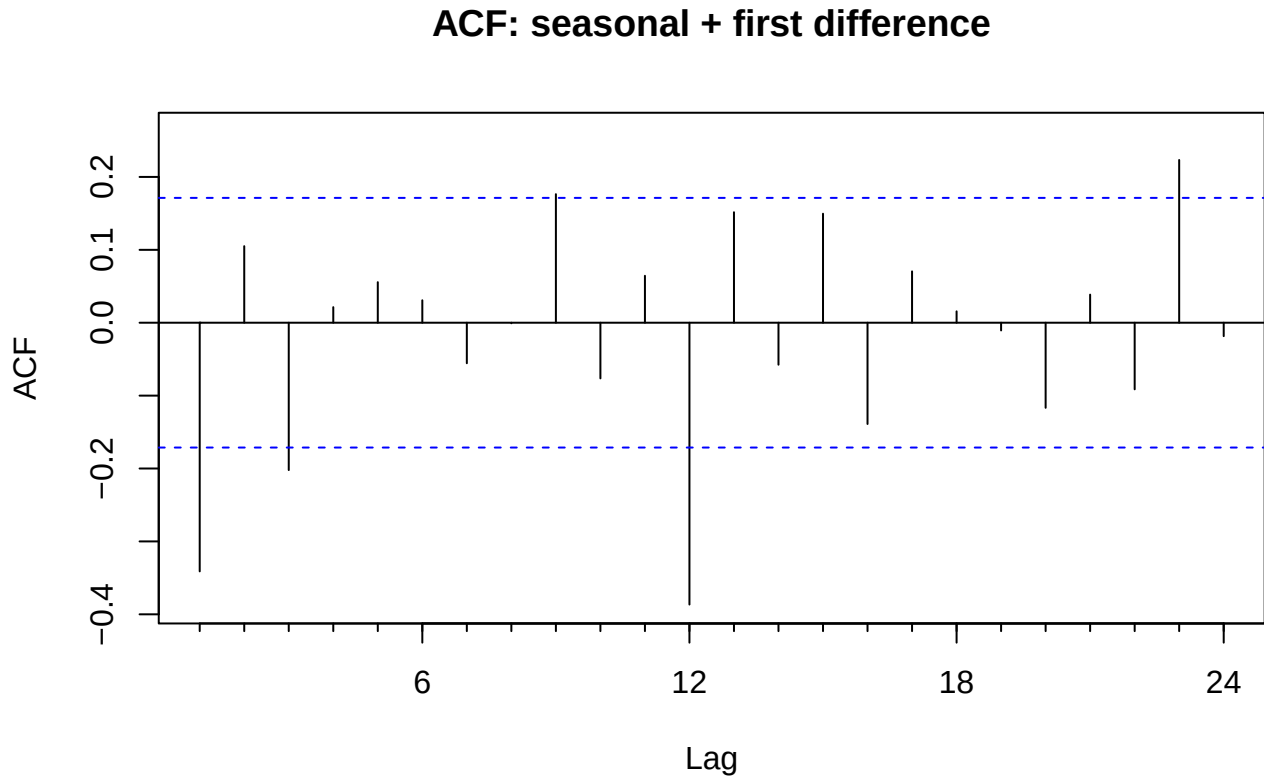
```
Acf(combined_diff_log_ap, main = "ACF: seasonal + first difference")
```

ACF: first difference

Hình 7: ACF của chuỗi sai phân bậc một.

ACF: seasonal difference

Hình 8: ACF của chuỗi sau sai phân mùa vụ.



Hình 9: ACF của chuỗi sau khi kết hợp sai phân mùa vụ và sai phân thường.

1. ACF của chuỗi log gốc có giảm chậm không?
2. Sau sai phân bậc một, ACF thay đổi thế nào?
3. Sau sai phân mùa vụ, các đỉnh tại độ trễ 12 có giảm không?
4. Chuỗi nào có vẻ phù hợp hơn để đưa vào mô hình ARMA/ARIMA?

2 Thực hành

2.1 Ví dụ mô phỏng dữ liệu giáo dục

Trong phần này, ta tạo một chuỗi mô phỏng số lượt truy cập LMS theo tháng. Chuỗi có xu hướng tăng, mùa vụ và nhiễu ngẫu nhiên. Thực hiện các bước tương tự trên và trả lời các câu hỏi bên dưới

1. Chuỗi LMS mô phỏng có xu hướng không?
2. Chuỗi có mùa vụ 12 tháng không?
3. Sau biến đổi log, phương sai có ổn định hơn không?
4. Sai phân thường có loại bỏ xu hướng không?
5. Sai phân mùa vụ có làm giảm tính lặp lại theo năm không?
6. Nếu chuẩn bị xây dựng mô hình SARIMA, em dự kiến chọn d và D như thế nào?

2.2 Bài tập thực hành 1.

Tự tạo một chuỗi mô phỏng số sinh viên đăng ký học phần theo tháng trong 5 năm. Chuỗi cần có:

- xu hướng tăng nhẹ;
- mùa vụ theo năm;
- nhiễu ngẫu nhiên.

Sau đó thực hiện:

1. Vẽ chuỗi gốc.
2. Lấy log nếu cần.
3. Sai phân bậc một.
4. Sai phân mùa vụ.
5. Vẽ ACF trước và sau sai phân.
6. Viết nhận xét khoảng 8–10 dòng.

2.3 Bài tập thực hành 2.

Sinh viên thu thập một bộ dữ liệu thực tế, ví dụ doanh số bán hàng theo tháng, số lượt truy cập website theo ngày, ... Sau đó, thực hiện các yêu cầu sau với bộ dữ liệu vừa thu thập được.

1. Giới thiệu ngắn về dữ liệu.
2. Vẽ chuỗi gốc.
3. Nhận xét xu hướng, mùa vụ và phương sai.
4. Thực hiện ít nhất hai phép biến đổi.
5. Thực hiện sai phân thường và sai phân mùa vụ nếu phù hợp.
6. Vẽ ACF trước và sau sai phân.
7. Đề xuất bước mô hình hóa tiếp theo.